
CSC 123 – Lab 06

The Die Class

Total Points: 10

Purpose

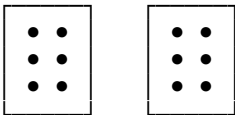
In this lab you will implement a reusable Java class named **Die**.

A die represents a random number generator with a fixed number of sides.
When rolled, the die returns a random integer between **1 and the number of sides**.

This class will later be used in a console program that simulates the game **Craps**, where two dice are rolled and displayed using ASCII graphics.

Example output from the future Craps driver:

```
[ r ] roll    [ n ] new game    [ q ] quit  >
```



```
rolled: 6 + 6 = 12
```

Your **Die** class will generate the values used by this program.

Required File

Submit only: `Die.java`

You may write your own test program while developing, but submit **only** `Die.java`.

Required Comment Header

Every `.java` file must begin with this header format

Example:

```
/* *****  
 * Name: Jane Student
```

```
* Course: CSC123
* Lab: Lab 08
* Date: 03/18/2026
* File: Die.java
* Description: A class representing a die with a variable
* number of sides that can be rolled.
*****/
```

Missing headers will result in point deductions.

Class Name

Create the class: `Die`

Required Instance Variables

Your class must contain **exactly two private fields**:

```
private int numSides;
private Random random;
```

Include the import:

```
import java.util.Random;
```

Constructors

Your class must implement **three constructors**.

1. Default Constructor

```
public Die()
```

Behavior:

- creates a **6-sided die**
- creates a **new Random object**

The Random object must be stored as a field and reused.

2. Constructor with Number of Sides

```
public Die(int numSides)
```

Behavior:

If `numSides < 1` print exactly:

```
Error: Number of sides must be at least 1. Using default of 6.
```

and the die must use **6 sides**.

Otherwise store the provided number of sides.

A **new Random object must also be created**.

Invalid values **must reset the die to 6 sides** rather than being ignored.

3. Constructor with Sides and Random Object

```
public Die(int numSides, Random random)
```

Behavior:

- use the **same validation rule** for `numSides`
- store the **provided Random object**

This constructor allows **deterministic testing of dice rolls** and will be used by automated testers.

Required Methods

roll()

```
public int roll()
```

Returns a random integer from:

1 to `numSides`

The `roll()` method must contain the statement:

```
return random.nextInt(numSides) + 1;
```

Your code **must include these comments explaining the logic**:

```
// nextInt(n) gives 0 to n - 1, so +1 shifts it to 1..n in an uniform discrete distribution
// using a provided Random lets test code control the sequence
```

Do NOT create a new Random object inside `roll()`.

getNumSides()

```
public int getNumSides()
```

Returns the number of sides on the die.

setNumSides(int numSides)

```
public void setNumSides(int numSides)
```

Behavior:

If `numSides < 1` print exactly:

```
Error: Number of sides must be at least 1. Using default of 6.
```

and set the die to **6 sides**.

Otherwise update the number of sides.

The **same message and behavior must be used in both the constructor and setter**.

The error message must appear exactly **two times in the source code**:

- once in a constructor
- once in `setNumSides(int)`

Do not create helper methods for validation.

toString()

Override `toString()`.

The output format **must be exactly**:

```
Die[sides=6]
```

Example:

```
Die[sides=20]
```

Spacing and capitalization must match exactly. The grader checks this.

Required Trace Comment

At the **bottom of the file**, include a block comment demonstrating how `roll()` works.

Example:

```
/*
Trace example:

If numSides = 6 and random.nextInt(6) returns 4,
then roll() returns 5.
*/
```

Example Test Code

Example testing program (**not submitted**):

```
Die d1 = new Die();
Die d2 = new Die(20);

System.out.println(d1);
System.out.println(d2);

System.out.println("Roll: " + d1.roll());
System.out.println("Roll: " + d2.roll());
```

Example output:

```
Die[sides=6]
Die[sides=20]

Roll: 4
Roll: 17
```

How This Will Be Used Later

Your **Die** class will be used by a **Craps program** that repeatedly prompts:

```
[ r ] roll    [ n ] new game    [ q ] quit
```

When the user rolls, two dice are rolled and displayed:

```
rolled: 6 + 6 = 12
```

The driver depends on your `roll()` method producing values in the correct range.

UML Diagram

```
-----
|           Die           |
|-----|
| - numSides : int       |
| - random : Random      |
|-----|
| + Die()                |
|-----|
```

```
| + Die(int) |
| + Die(int, Random) |
| + roll() : int |
| + getNumSides() : int |
| + setNumSides(int) |
| + toString() : String |
-----
```

Requirements Summary

Your class must:

- contain private fields
- implement all three constructors
- reset invalid side counts to **6**
- print the required error message when invalid
- implement `roll()`
- implement `getNumSides()`
- implement `setNumSides(int)`
- override `toString()` exactly
- include the required header
- include the required comments
- include the trace comment
- follow proper formatting

Do NOT create additional helper methods.

All validation logic must appear directly in the constructors and setter.

Restrictions

For this lab **do not use**:

- arrays
- inheritance
- collections
- GUI libraries
- file input/output

Focus on basic class design.

AI Code Critic (LLM Code Review)

You must also include with your `Die.java`, an **AI review summary**.

Prompt example:

Analyze this Java class for correctness, style issues, and edge cases.
Suggest improvements without rewriting the whole program.

Include the AI Feedback Summary (3–5 bullet points) and whether you accepted or rejected the suggestions.
Add this to bottom of your class file (above Required Trace Comment)

Example:

```
/*  
AI Review Summary:  
1.  
2.  
3.  
  
Student Response:  
...  
*/
```

Learning outcomes:

- Critical evaluation of AI output
- Recognizing hallucinations or incorrect advice
- Software engineering reflection

Grading (10 points)

Category

Correct fields
Constructors
Roll() method
Getter/setter with validation
Header + formatting + toString()
AI code review

Deliverable

Submit:

Die.java

Your file must:

- compile
 - include the required header
 - match the required method names
 - follow proper formatting
-

Next Class: Short Explanation

In the next class meeting you may be asked to briefly explain parts of your implementation.

Be prepared to explain the following:

1. **How `roll()` avoids returning 0.**
Why `nextInt(numSides) + 1` produces values from **1 to numSides**.
2. **Why the constructor `Die(int numSides, Random random)` exists.**
How allowing a `Random` object to be provided enables **controlled testing** of dice rolls.
3. **What happens when `numSides` is invalid.**
Why the class prints the error message and resets the die to **6 sides**.
4. **Why `Random` is stored as a field.**
Explain the difference between:
 - storing a `Random` object in the class
 - creating a new `Random` object inside `roll()`

Students may be randomly selected to explain one of these topics.